**SafetySense**
CALLS FOR HELP WHEN YOU CAN'T

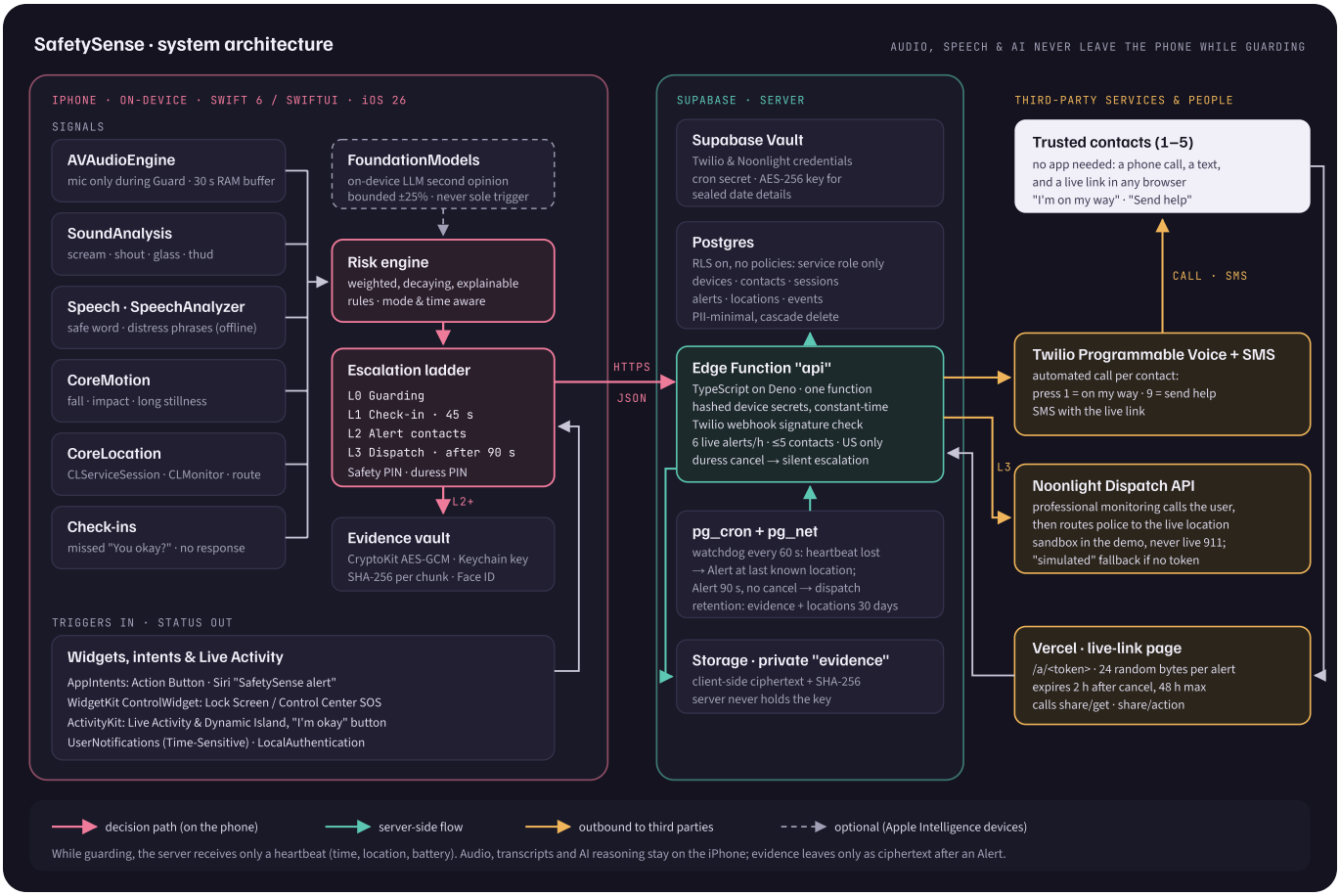
ATTACHMENT 4 OF 5 · PROPOSED TECHNICAL ASPECTS

# On-device intelligence, a small server, real calls

The iPhone does all the sensing and deciding. A minimal Supabase backend keeps a heartbeat, calls contacts through Twilio, and hands off to dispatch through the Noonlight API.

iOS **Swift 6 · SwiftUI · iOS 26** Server **Supabase Edge Functions · Postgres**
Services **Twilio · Noonlight · Vercel**

## 01 Architecture diagram




Everything in the pink panel runs on the phone. The server sees a heartbeat while guarding, and encrypted evidence only after an Alert. A full-page, larger copy is on the last page.

**0**
third-party iOS SDKs. Only Apple frameworks, with no analytics or ads.

**3**
on-device models: sound events, speech, and an optional LLM second opinion.

**60 s**
heartbeat. The server raises an alert by itself if the phone goes dark.

**136**
automated tests (Swift Testing), plus a dry-run backend smoke test.

## 02 On-device components (Apple frameworks)

| FRAMEWORK / API | WHAT SAFETYSENSE USES IT FOR |
|---|---|
| **AVAudioEngine, AVAudioSession** | Background microphone capture, only during a Guard session. Audio is held in a fixed 30-second ring buffer in RAM. |
| **SoundAnalysis** | Apple's built-in sound classifier (`SNClassifySoundRequest`, about 300 labels). We watch 10 of them (screaming, shout, yell, crying, gasp, slap, thud, glass breaking, door slam, gunshot), each with its own confidence threshold. |
| **Speech:** `SpeechAnalyzer` | iOS 26 on-device transcription that spots the safe word and distress phrases. It falls back to `SFSpeechRecognizer` with `requiresOnDeviceRecognition`, so nothing goes to a server. |
| **FoundationModels** | Apple's on-device LLM (`LanguageModelSession` with a `@Generable` structured answer and greedy sampling). It adjusts the risk score by at most ±25% and can't be the sole trigger. Phones without Apple Intelligence use rules only. |
| **CoreMotion** | Impact or fall (a spike, then stillness), erratic movement followed by long stillness, or the phone being dropped. |
| **CoreLocation** | `CLServiceSession` and `CLLocationUpdate` for live location, and `CLMonitor` geofences for arrival checks and route deviation. |
| **AppIntents, WidgetKit** | Action Button and Siri trigger, and a `ControlWidget` SOS for the Lock Screen and Control Center. These work without opening the app. |
| **ActivityKit** | Live Activity and Dynamic Island with the check-in countdown and an "I'm okay" button. |
| **UserNotifications, BackgroundTasks** | Time-Sensitive check-in alerts. `BGTaskScheduler` flushes the offline outbox. |
| **CryptoKit, Security (Keychain)** | AES-256-GCM evidence sealing with a this-device-only Keychain key. SHA-256 tamper hashes. Hashed PINs. |
| **LocalAuthentication, ContactsUI, MapKit** | Face ID for the Vault and ending a session, picking trusted contacts, and the in-app map. |

### The risk engine (our code)

Each signal adds weighted points that fade over time. Weights depend on the mode (Night Out, Date, Walk, Ride) and the time of day, and the user picks Relaxed, Balanced or Vigilant sensitivity. The engine is deterministic and unit-tested, and every escalation carries a plain-language reason ("Scream, then no movement"). A "playful context" rule halves shouting when laughter or cheering is clearly present, which cuts party false alarms.

## 03 Server, APIs and third-party services

| SERVICE | ROLE | DATA IT RECEIVES |
|---|---|---|
| **Supabase Edge Function** `api` | One TypeScript/Deno function with every route (table below). It does its own authentication and calls the services below. | Device ID, hashed secret, session and alert metadata |
| **Supabase Postgres** | Devices, contacts, sessions, alerts, locations and events. Row-level security is on with no policies, so only the server role can read or write. | See the data inventory in Attachment 5 |
| **Supabase Vault** | Encrypted secret store: Twilio and Noonlight credentials, the cron secret, and the AES-256 key for sealed date details. | Secrets only |
| **pg_cron + pg_net** | Runs a watchdog every minute (heartbeat lost → Alert, 90 s at Alert → dispatch) and an hourly retention job (evidence and locations after 30 days, alerts after 90 days, test alerts after 48 h). An `account/delete` route erases a device's data on request. | None (internal) |
| **Supabase Storage** | Private `evidence` bucket. | Ciphertext and SHA-256 only |
| **Twilio Programmable Voice + SMS** | An automated call to each contact ("press 1 if you're on your way, 9 to send help"), verified with signed webhooks, plus an SMS with the live link. | Contact phone, user's first name, alert reason, link |
| **Noonlight Dispatch API** | Professional monitoring: a dispatcher texts and calls the user to verify, then, with no PIN or answer, passes the live location to the local 911 center (`POST /dispatch/v1/alarms`, plus location and status updates). | Name, phone, location (Level 3 only) |
| **Vercel (static hosting)** | The contact live-link page `/a/<token>` and the landing page. Plain HTML and JS with Leaflet 1.9.4 and OpenStreetMap tiles. | Nothing stored. The page reads the API with its token |

### API surface (all `POST` JSON)

| ROUTE | AUTH | PURPOSE |
|---|---|---|
| `register` | public key | Creates a device and returns `device_id` and `device_secret` once. The server keeps only a SHA-256 of the secret. |
| `contacts/set` | device | Replaces the trusted-contact list (at most 5, US numbers only). |
| `session/start · heartbeat · end` | device | Starts a Guard session (mode, check-in interval, grace period, sealed date details), sends a 60 s heartbeat with location and battery, and ends the session cleanly. |
| `alert/create · escalate · cancel` | device | Level 2/3 alerts with the trigger and reason, escalation to dispatch, and cancel (`duress:true` silently escalates but answers "cancelled"). |
| `alert/location · status` | device | Live location during an alert, and the contact and dispatch timeline shown in the app. |
| `evidence/upload` | device | An encrypted evidence chunk with its sequence number and SHA-256. |
| `account/delete` | device | "Delete everything": erases the device's contacts, sessions, alerts, locations, events and evidence files, and revokes its secret. |
| `share/get · share/action` | link token | The live-link page: status, location, battery, sealed details while the alert is active, and "on my way", "called 911" or "send help". |
| `twilio/* · cron/*` | signature · secret | Call TwiML, keypress handling and SMS delivery receipts (Twilio-signed), the minute-by-minute watchdog and the hourly retention job. |

## 04 Key flows

### A scream, then silence

1. `0 s` SoundAnalysis flags screaming. Then CoreMotion sees stillness.
2. The risk engine crosses its threshold, and a Level 1 check-in starts with a haptic and the Live Activity.
3. `+45 s` No answer. Level 2: the app seals the 30 s buffer as evidence and calls `alert/create`.
4. The server calls and texts each contact through Twilio. Contacts open the live link.
5. `+90 s` No PIN. The watchdog or the app escalates to Noonlight, which calls the user and then dispatches.

### The phone goes dark

1. During a session the app sends a heartbeat every 60 s. Missed sends wait in an offline outbox.
2. The phone is smashed, switched off or put in airplane mode, and heartbeats stop without "End session".
3. After the grace period, pg_cron's watchdog creates a Level 2 `heartbeat_lost` alert at the last known location.
4. Contacts are called, and dispatch follows after 90 s unless a contact resolves it.
5. Evidence uploaded before the phone went dark is already safe on the server, encrypted.

## 05 Engineering quality

- **Swift 6** language mode with complete strict concurrency, SwiftUI, and an XcodeGen project. The build has zero warnings.
- **Protocol-based services** (sensing, risk, escalation, backend, vault) with demo and mock versions, so tests and demo videos never touch the network or place calls.
- **136 Swift Testing tests** covering the risk engine, detectors, escalation timing, the duress path, vault encryption and tamper checks, and the backend contract. A shell smoke test runs the whole server flow in dry-run mode.
- **Resilience:** an offline outbox for heartbeats and alert locations, one live alert per session (repeat calls reuse it), and server-side escalation that doesn't depend on the phone surviving.

## 06 What's real today, and what comes next

| COMPONENT | STATUS | NOTES |
|---|---|---|
| **On-device detection, risk engine, ladder, PINs** | BUILT | Runs in the iOS 26 simulator. The iPhone 16e demo device adds the Action Button and Apple Intelligence. |
| **Backend, watchdog, live-link page** | LIVE | Deployed on Supabase and Vercel. |
| **Twilio voice calls to contacts** | WORKING | Trial account: calls reach verified numbers only, after Twilio's trial message. Production needs a paid account. |
| **Twilio SMS** | BLOCKED ON TRIAL | US SMS needs A2P 10DLC registration (error 30034). Failures are recorded and don't stop the calls. |
| **Noonlight dispatch** | SANDBOX / SIMULATED | Uses the sandbox only (`api-sandbox.noonlight.com`; the token is set). Dry runs, and any run without a token, record a clearly labelled "simulated dispatch" instead; every demo alert so far has been a dry run. It never contacts live 911. |
| **Next** | ROADMAP | App Attest on alert routes, Sign in with Apple for account recovery, the Critical Alerts entitlement, Apple Watch, an Android version, and production Noonlight after legal review. |

**APPENDIX** The architecture diagram at full size (rotated). Source: `design/architecture.svg`.

